MDPI

Article

Mathematical Modeling of Large Language Model Maintenance: A Stochastic Optimal-Control Framework for Retraining Scheduling, Knowledge Retention, and Alignment Preservation

Ook Lee

Department of Information Systems, Hanyang University, Seoul 04763, Republic of Korea; ooklee@hanyang.ac.kr

Abstract

Large language models (LLMs) are deployed as long-lived software artifacts whose usefulness degrades as the world drifts away from their training distribution, yet they are still largely maintained by fixed calendars and ad hoc judgment. This paper treats a deployed LLM as a deteriorating system and casts its maintenance—when, how, and how much to update it—as a constrained stochastic optimal-control problem. Loss of freshness is modeled as a shock-and-wear process driven by drift events, update-induced catastrophic forgetting as a Fisher-information-weighted quadratic penalty interpreted as imperfect maintenance, and update cost as scaling with model and data size. Scheduling is formulated as an average-cost Markov decision process whose optimal policy, under stochastic-monotonicity and submodularity conditions, escalates monotonically through a graduated menu of maintenance actions. Retrieval-augmented generation is characterized as a feedback compensator, with a closed-form condition under which blending it with parametric maintenance dominates either alone. A square-root law links update cost, drift intensity, and the optimal maintenance interval, and lower forgetting per event is shown to warrant more frequent maintenance. A case study on a public pretrained model confirms affine staleness growth, recovers a retrieval-absorbed drift share equal to retrieval coverage, and shows that unreplayed low-rank adaptation can be dominated by taking no action.

Keywords: large language models; model maintenance; catastrophic forgetting; concept drift; condition-based maintenance; continual learning; Markov decision processes; renewal theory

Received:
Revised:
Accepted:
Published:

Citation: Lee, O. Mathematical Modeling of Large Language Model Maintenance. *Electronics* **2026**, *1*, 0. https://doi.org/

Copyright: © 2026 by the author. Submitted to *Electronics* for possible open access publication under the terms and conditions of the Creative Commons Attri- bution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

1.1. Motivation

A large language model (LLM) is, at the moment its training run ends, already beginning to become obsolete. The corpus on which it was trained is a snapshot of a world that continues to change: new facts are established, old facts are superseded, vocabulary and norms shift, and the population of tasks users bring to the model evolves. At the same time, the model's own behavior—the outputs it produces under reinforcement learning from human feedback (RLHF) or other post-training procedures—is itself a target that organizations wish to keep aligned with changing safety, policy, and product requirements. The practical response to this obsolescence is maintenance: some combination of full retraining, continual fine-tuning, targeted knowledge editing, and retrieval augmentation, applied at some cadence, to keep the deployed model acceptably close to a moving target.

From the standpoint of reliability engineering, a deployed LLM is a deteriorating system. Its degradation index is not physical wear but the accumulated divergence between what the model was trained on and what it is now asked to do; its maintenance actions are not part replacements but parameter updates; and, as with imperfect maintenance of physical equipment [1], every maintenance action carries a risk of damaging the system it is meant to restore. The condition-based maintenance (CBM) literature has developed a mature toolkit—degradation processes, control-limit policies, renewal-reward optimization—for exactly this structure [2–4]. This paper transfers that toolkit to LLMs.

Maintenance of statistical models is not a new problem; it is the subject matter of the concept-drift and machine-learning operations (MLOps) literatures for classical machine-learning models [5,6]. What is new with LLMs is the combination of (a) extremely high update costs that scale with model and data size in a manner now well characterized by empirical scaling laws [7,8]; (b) an update mechanism—gradient-based fine-tuning of an enormous, highly entangled parameter vector—that is well known to induce catastrophic forgetting of previously acquired competence [9–11]; and (c) a growing menu of update modalities other than full retraining, including parameter-efficient fine-tuning (PEFT), targeted knowledge editing [12,13], and retrieval-augmented generation (RAG) [14], each with a different cost/forgetting/freshness profile. Practitioners must therefore make a compound decision—whether to update, how much to update, and by which mechanism to update—repeatedly, under uncertainty about how quickly the world is drifting.

Despite the practical urgency of this decision problem, it is typically addressed heuristically: fixed retraining calendars, ad hoc drift alarms, or qualitative judgment calls about how much fine-tuning is too much. The continual-learning community has produced a rich set of algorithms for mitigating forgetting during an update [9,11,15], and the concept-drift community has produced a rich set of triggers for deciding when a classical model should be retrained [5,6]. What is comparatively underdeveloped is a unifying mathematical account that treats the decision of whether/how/how-much to maintain an LLM as a single constrained optimization problem, in which freshness decay, forgetting risk, and cost are all first-class, jointly optimized quantities rather than separately tuned heuristics.

1.2. Problem Statement

This paper addresses the following question: Can the maintenance of a deployed large language model be formulated as a well-posed mathematical optimization problem, and if so, what is the structure of its optimal solution? We decompose this question into three sub-problems.

1. *Freshness modeling.* How should the decay of a deployed model's task-relevant accuracy be represented mathematically as a function of time and of the (unobserved) rate at which the world changes?
2. *Forgetting-constrained update modeling.* Given a decision to update the model, how should the trade-off between absorbing new information and retaining old competence be formalized, and what does this imply about the feasible set of update magnitudes?
3. *Scheduling under cost.* Given models for freshness decay and forgetting, what is the cost-minimizing long-run policy for deciding when to invoke which maintenance mechanism, and what qualitative shape does that policy take?

1.3. Contributions

This paper makes five contributions.

- It proposes a shock-and-wear model of knowledge staleness, driven by a marked point process of drift events, that generalizes the deterministic decay curves implicit in

empirical retraining-cadence studies [16] to a setting with random drift-event arrivals and random drift magnitudes, in the spirit of the concept-drift literature [5] and of shock-and-wear degradation models in reliability theory [3].

- It reformulates elastic weight consolidation (EWC) [9] and related Fisher-information regularization schemes as an explicit constraint on the feasible parameter-displacement set during an update, which allows the amount of forgetting purchased by an update to be expressed as a scalar that enters the maintenance cost function directly—an imperfect-maintenance effect in the sense of [1].
- It derives, via the renewal-reward theorem and a discrete-time Markov decision process (MDP) formulation, sufficient conditions under which the optimal maintenance policy is monotone (threshold-structured) in the degradation state, connecting the LLM maintenance problem to classical replacement theory [17] and to recent results on resource allocation under concept drift [18].
- It formalizes retrieval-augmented generation as a feedback compensator acting on the observed staleness signal without displacing model parameters, and derives a closed-form condition under which blending retrieval and parametric updates dominates either mechanism alone.
- It instantiates the measured inputs of the framework—staleness growth, retrieval-absorbed share, and per-mechanism cost, effectiveness, and forgetting—on a public pretrained model, and reports both confirmations of the model's assumptions and one measured violation of them.

1.4. Paper Organization

Section 2 reviews the relevant literature. Section 3 gives a formal definition of the LLM maintenance problem and states the assumptions maintained throughout. Section 4 develops the stochastic model of knowledge decay and derives the renewal-reward-optimal fixed-threshold retraining policy. Section 5 develops the Fisher-information-constrained model of update-induced forgetting. Section 6 integrates both components into an MDP and proves the monotone structure of the optimal policy. Section 7 treats retrieval augmentation as a compensating controller. Section 8 provides a numerical illustration and a case study with a public pretrained model. Section 9 draws managerial implications, Section 10 discusses limitations, and Section 11 concludes.

2. Background and History

2.1. Historical Development

The problem of deciding when to maintain a deteriorating system predates machine learning. Reliability theory formalized replacement and maintenance policies for equipment whose condition degrades over time, including age- and block-replacement policies that trade the cost of intervention against the cost of failure [17], and later condition-based policies that act on an observed degradation index rather than on age alone [2,4]. Renewal theory supplies the tools for analyzing such policies over an infinite horizon, most notably the renewal-reward theorem [19], and the theory of Markov decision processes extends them to settings with several possible actions and a controlled state [20,21]. In machine learning, the analogous problem emerged when models were deployed on data streams whose distribution changes over time, giving rise to the concept-drift literature and its retraining triggers [5]. The arrival of transformer-based language models [22] and their scaling to few-shot learners [23] changed the economics of the problem: retraining became vastly more expensive, while updates to a single, highly entangled parameter vector became more likely to erase prior competence [9,11]. The remainder of this section reviews the four strands of work on which this paper builds.

2.2. Catastrophic Forgetting and Continual Learning in LLMs

Catastrophic forgetting—the tendency of a neural network trained sequentially on new data to overwrite the representations needed for earlier tasks—has been a central concern of continual learning since its formulation in the connectionist literature and remains so in the LLM era, where it acquires new manifestations tied to instruction tuning, alignment, and continual pre-training [11,15]. Shi et al. [11] survey the space of mitigation techniques, organizing them into *replay-based* methods, which rehearse a subset of prior data or synthetic surrogates during an update; *regularization-based* methods, which penalize movement of parameters deemed important to earlier tasks; and *architecture-expansion* methods, which allocate new capacity for new tasks while freezing capacity associated with old ones. Wu et al. [15] propose a complementary categorization organized around the stage of the LLM lifecycle at which continual adaptation occurs—continual pre-training, continual instruction tuning, and continual alignment—and note that the sheer cost of retraining LLMs from scratch makes incremental adaptation not merely convenient but close to necessary at scale.

Empirically, Luo et al. [10] conduct a systematic study of forgetting during continual instruction tuning of LLMs ranging from one to seven billion parameters, evaluating forgetting along the axes of domain knowledge, reasoning ability, and reading comprehension. Their central and somewhat counter-intuitive finding is that forgetting intensifies with model scale within the range studied, which the authors attribute to larger models having more capability to lose in the first place; they also find that decoder-only architectures forget less than encoder–decoder architectures of comparable scale, and that broad instruction tuning prior to a narrower fine-tuning stage can attenuate subsequent forgetting. These findings matter for a mathematical theory of maintenance because they imply that the forgetting-penalty term in any maintenance cost function cannot be treated as scale-invariant: larger, more capable, and more heavily aligned models are not automatically more robust to further updates, and in some regimes may be less so.

The dominant mathematical device for controlling forgetting during an update is EWC, introduced by Kirkpatrick et al. [9]. EWC casts sequential learning as approximate Bayesian filtering: the posterior over parameters after training on an initial task is approximated, via a Laplace approximation around the maximum-a-posteriori solution, by a Gaussian whose precision matrix is the diagonal of the Fisher information matrix evaluated at that solution. When a subsequent task's loss is optimized, this approximate posterior is folded in as a quadratic penalty on the displacement of each parameter from its prior value, weighted by that parameter's estimated importance to the earlier task. Follow-up work has examined the penalty critically. Huszár [24] showed that accumulating a separate quadratic penalty per task, as in the original formulation, is not the correct Bayesian recursion for more than two tasks, and Kunstner et al. [25] showed that the empirical Fisher, which is commonly used in practice to compute the importance weights, can differ substantially from the true Fisher, so the weights can misjudge which parameters truly matter to prior competence. For the purposes of this paper, EWC and its relatives are important not for their algorithmic details but for what they reveal structurally: any update to a deployed LLM can, in principle, be described as the solution to a constrained optimization problem, in which the constraint set is defined by an allowable radius of parameter displacement measured in a Fisher-information-weighted metric. This is the formal device adopted in Section 5.

2.3. Knowledge Editing and Retrieval Augmentation

An alternative to full or partial retraining is to modify a narrow slice of a model's stored knowledge directly. Wang et al. [12] survey knowledge-based model editing (KME),

formalizing the goal as locating and rewriting a specific fact within a pre-trained model's parameters while leaving unrelated behavior unaffected—a substantially more surgical intervention than gradient-based fine-tuning over a broad dataset. Zhang et al. [13] extend this survey with a benchmark and a systematic study of where factual knowledge is localized within transformer layers, motivated explicitly by the same economic pressure this paper takes as a starting assumption: that naively retraining an LLM every time a fact becomes outdated is computationally prohibitive, and that a lighter-weight editing mechanism is therefore economically attractive whenever the update in question is narrow in scope.

A structurally different response to knowledge staleness is RAG. Gao et al. [14] survey the paradigm, in which a model retrieves passages from an external, independently updatable corpus at inference time and conditions its generation on that retrieved evidence, rather than relying solely on knowledge encoded in its parameters at training time. Because the corpus can be updated continuously and cheaply relative to the cost of retraining the model itself, RAG effectively separates the freshness of the information a model can draw upon from the currency of its parameters. This separation is central to Section 7, where we argue that RAG is best modeled not as a substitute for the maintenance decision but as an additional control input that acts on the observed staleness signal without touching the parameter vector, and therefore without incurring forgetting risk.

2.4. Concept Drift and Retraining Scheduling in Classical MLOps

The question of *when* to retrain a deployed statistical model, as opposed to *how* to retrain it, has a long history in the concept-drift literature. Lu et al. [5] provide a comprehensive review, distinguishing between sudden, gradual, incremental, and recurring drift patterns and cataloguing detection statistics used to trigger retraining. Two retraining philosophies recur throughout this literature and correspond exactly to the age-based and condition-based dichotomy of reliability theory [4]: *periodic* retraining, in which the model is refreshed on a fixed calendar regardless of observed performance, and *informed* (or triggered) retraining, in which a drift-detection statistic determines the retraining time endogenously. A companion design axis concerns which data are used at retraining time—a sliding-window approach that discards old data in favor of recent observations, versus a full-history approach that accumulates all data seen to date—each of which implies a different bias–variance and forgetting profile.

Mahadevan and Mathioudakis [6] formalize the retraining-timing decision explicitly as a cost-minimization problem over a data stream, proposing a cost-aware retraining algorithm that adapts to the joint statistics of drift severity and retraining expense, and showing empirically that a cost-aware policy dominates naive drift-triggered baselines both in accuracy and in the number of retraining events incurred. Beytur et al. [18] study the underlying resource-allocation problem analytically for a provider that trains models and distributes them to clients that cannot retrain locally. Modeling the durations of stationary concept regimes as random variables, they show that the optimal training policy depends critically on the aging property of the concept-duration distribution: under sudden concept changes they derive optimal budget-constrained training policies when durations have decreasing mean residual life, and show that intuitive heuristics are provably suboptimal when durations have increasing mean residual life. This is one of the few treatments in the literature that derives, rather than assumes, the structural form of an optimal retraining policy, and Sections 4.3 and 6 build on the renewal-theoretic and optimal-control tools used there. A retrospective clinical study by Williams et al. [16] offers a complementary, purely empirical data point: in a five-year rolling simulation of a paediatric emergency-department admission model, nine retraining cadences ranging from weekly to triennial,

plus a static model, were compared directly. Fortnightly and monthly retraining were non-inferior to weekly retraining, with monthly retraining requiring about one quarter of the weekly computational burden, whereas longer intervals showed progressive degradation of calibration—an observation consistent with a diminishing-returns relationship between retraining frequency and maintained accuracy, a relationship this paper makes explicit in Section 4.

2.5. Cost Structure: Scaling Laws and Alignment

Any maintenance policy is a policy over costly actions, and the cost of the dominant action—retraining or fine-tuning—is itself now understood with unusual empirical precision. Kaplan et al. [8] establish that the pre-training loss of a transformer language model follows a power law in model size, dataset size, and training compute across many orders of magnitude, and use the approximation that training costs about $6ND$ floating-point operations (FLOP) for N parameters and D tokens. Hoffmann et al. [7] refine this picture by jointly optimizing model size and token count for a fixed compute budget, showing that many contemporary models had been substantially over-parameterized relative to the volume of data on which they were trained. The FLOP accounting is used in Section 3 to translate an update's size and scope into an expected monetary or compute cost.

A further cost dimension specific to LLMs is alignment maintenance. Ouyang et al. [26] show that fine-tuning a pre-trained language model with human feedback substantially improves the model's helpfulness and truthfulness as judged by human evaluators, to the point that a much smaller aligned model can be preferred to a much larger unaligned one. Their study also documents an *alignment tax*: naive reinforcement fine-tuning degrades performance on some public NLP benchmarks relative to the base model, a cost the authors mitigate by mixing the original pre-training objective into the reinforcement-learning update. This is a direct, empirically documented instance of the stability–plasticity trade-off that Section 5 formalizes: the alignment update is itself a maintenance action that purchases improved conformity to current human preferences at some risk of degrading previously acquired general capability, and the mitigation strategy—blending old and new objectives—plays the same structural role as the Fisher-information-weighted penalty used in EWC.

2.6. Synthesis and Gap

Table 1 summarizes how the four strands of literature reviewed above map onto the components of the maintenance problem formalized in Section 3. Continual learning and knowledge editing address *how* an update should be executed once a decision to update has been made; concept-drift research addresses *when* an update should be triggered for classical, comparatively cheap-to-retrain models; scaling-laws and alignment research quantify *how costly* an update is once its size and scope are fixed. No single strand jointly optimizes the when, how much, and by which mechanism decisions under a shared cost objective that includes both forgetting risk and monetary cost, which is precisely the integration this paper undertakes.

Table 1. Literature strands mapped to the components of the maintenance problem.

Strand	Representative Work	Question Addressed	What It Leaves Open
Continual learning and forgetting	[9–11,15]	How to execute an update without losing prior competence	When to update; monetary cost
Knowledge editing and retrieval	[12–14]	How to update narrowly, or without touching parameters	Scheduling; cost model
Concept drift and retraining scheduling	[5,6,16,18]	When to trigger an update	Developed for cheap-to-retrain classical models
Scaling laws and alignment	[7,8,26]	How costly an update is	Retention and scheduling decisions
Reliability and CBM	[1,3,4,17]	Optimal maintenance of a deteriorating system	Application to learned models with update-induced damage

3. Formal Definition of LLM Maintenance

Table 2 summarizes the notation used throughout the paper.

Table 2. Summary of notation.

Symbol	Meaning
$\theta_t \in \mathbb{R}^p$	Parameter vector of the deployed model at time $t \geq 0$; θ_0 is the initial checkpoint
$t_k,\ t_k^-,\ t_k^+$	Time of the k-th maintenance event and the instants just before and after it
Δ_k	Parameter displacement of the k-th maintenance action, $\theta_{t_k^+} = \theta_{t_k^-} + \Delta_k$
$\mathcal{C}_t$	External retrieval corpus at time t
$w_t \in \mathcal{W}$	World state (distribution of facts, tasks, and norms) at time t
$u(\theta, x)$	Bounded per-example utility of parameters θ on example x
$A(\theta, w)$	Task-relevant accuracy of θ against world state w
$S(t)$	Staleness: accuracy lost to world drift since t_k
$G(\Delta)$	Forgetting: accuracy lost on the pre-update world because of Δ
$X(t)$	Degradation index, $X(t) = S(t) + G(\Delta_k)$ on $(t_k, t_{k+1}]$
$c(\Delta)$	Operational cost of a maintenance action; c_r full retrain; c_{RAG} retrieval upkeep per unit time
$N_{\mathrm{eff}},\ p_b,\ D_k$	Trainable parameters, parameters traversed by back-propagation, and update tokens
κ	Conversion factor from FLOP to monetary cost
$\alpha,\ \beta$	Loss-rate coefficient on degradation; one-off cost per unit of forgetting
J	Long-run average maintenance cost rate
(τ_i, M_i)	Arrival time and magnitude of the i-th drift event; $M_i \sim F_M$ with mean μ_M
λ	Drift-event arrival rate (Poisson case)
$F_\tau,\ h_\tau(\cdot)$	Inter-arrival distribution and hazard rate of drift events
η	Background drift-leak rate
$\Lambda(v)$	Renewal function: expected number of drift events in an interval of length v
γ	Effective staleness growth rate, $\gamma = \eta + \lambda\mu_M$
$s,\ s^\star$	Degradation threshold that triggers maintenance, and its optimal value
$T(s),\ T^\star$	Cycle length under threshold s, and its optimal value
F	Diagonal Fisher information matrix of the pre-update model on the old world
$G_{\max}$	Organizational tolerance for forgetting per update
$P(\Delta)$	Plasticity (staleness reduction) delivered by Δ
$x_t = (X_t, d_t)$	MDP state: discretized degradation index and drift-hazard regime
$\mathcal{A},\ a_t$	Ordered set of parametric maintenance actions; action at epoch t
$\rho(a),\ \xi_{t+1}$	Degradation-reduction fraction of action a; random increment in an epoch
$q(k \mid x, a)$	Tail probability $\Pr(X_{t+1} \geq k \mid X_t = x, a_t = a)$
$r(x, a),\ h(\cdot),\ g$	One-step cost, relative value function, optimal average cost
$\phi(\cdot),\ \bar{\phi}$	Staleness reduction attributable to retrieval, and its average rate
$\tilde{S}(t)$	Effective staleness under retrieval augmentation

3.1. The Deployed Model as a Time-Indexed Object

Let $\theta_t \in \mathbb{R}^p$ denote the parameter vector of a deployed LLM at time $t \geq 0$, with θ_0 the parameters produced by the initial training run. Between maintenance events, θ_t is held fixed. A maintenance event at time t_k replaces $\theta_{t_k^-}$ with $\theta_{t_k^+} = \theta_{t_k^-} + \Delta_k$, where Δ_k is the parameter displacement induced by the k-th maintenance action; a full retraining corresponds to Δ_k that is unconstrained in norm (and typically very large), a knowledge edit or parameter-efficient fine-tune corresponds to a Δ_k that is sparse or low-rank, and a retrieval-augmentation event corresponds to $\Delta_k = 0$ together with an update to an external, independently indexed corpus $\mathcal{C}_t$.

3.2. Freshness and Staleness

We define an (unobservable in general, but statistically estimable) world state $w_t \in \mathcal{W}$ evolving as a stochastic process representing the distribution of facts, tasks, and behavioral norms the model will be asked to handle at time t. The model's task-relevant accuracy, or *freshness*, is a functional

$$A(\theta_t, w_t) = \mathbb{E}_{x \sim w_t}[u(\theta_t, x)], \tag{1}$$

where $u(\theta, x)$ is a bounded per-example utility (for example, an indicator of correctness, a human-preference score, or a negative loss). *Staleness* at time $t \in (t_k, t_{k+1}]$ is defined as

$$S(t) = A\left(\theta_{t_k^+}, w_{t_k}\right) - A\left(\theta_{t_k^+}, w_t\right), \tag{2}$$

the accuracy the deployed model would have retained had the world not moved, minus its accuracy against the current world. $S(t)$ is non-negative in expectation whenever w_t drifts away from w_{t_k} on average, and $S(t_k^+) = 0$ by construction. Staleness is the quantity that continual-learning, knowledge-editing, and retrieval-augmentation interventions all ultimately aim to reduce. Because u is bounded, $S(t)$ is bounded above by the range of A; the affine growth models of Section 4 are therefore local approximations valid while staleness is well below that bound.

3.3. Forgetting

Whenever a maintenance action induces a nonzero displacement Δ_k, it risks degrading accuracy on the portion of the world distribution already mastered, an effect distinct from, and in the worst case additive to, staleness. We define the *forgetting* incurred by an update as

$$G(\Delta_k) = A\left(\theta_{t_k^-}, w_{t_k^-}\right) - A\left(\theta_{t_k^-} + \Delta_k, w_{t_k^-}\right), \tag{3}$$

the drop in accuracy against the pre-update world distribution caused solely by the parameter displacement, holding the world fixed. $G(\Delta_k) > 0$ is the empirically documented phenomenon studied in [9–11], and it is a function of both the magnitude and the direction of Δ_k relative to directions of high curvature in the pre-update loss landscape, a dependency made explicit in Section 5. In reliability terms, G is the damage inflicted by an imperfect maintenance action [1].

3.4. The Degradation Index

Staleness (2) and forgetting (3) are measured against different world states, so neither alone describes the condition of the model after an imperfect update. We therefore define the *degradation index*

$$X(t) = S(t) + G(\Delta_k), \qquad t \in (t_k, t_{k+1}], \tag{4}$$

which starts at $X(t_k^+) = G(\Delta_k)$ after the k-th maintenance event and grows as the world drifts. Adding the two terms presumes that accuracy lost on the old world and accuracy lost to drift are commensurable and additive, which is part of Assumption A1 below. In the case study of Section 8.4, X is measured directly as the sum of the error on a drifting evaluation suite and the error on a static retention suite. The degradation index X plays the role in this paper that a wear or damage process plays in classical reliability theory [3]; when an action induces no forgetting, $X = S$.

3.5. Cost

Each maintenance action carries an operational cost. Following the standard FLOP accounting of scaling-law analyses [7,8], a forward pass costs about two FLOP per parameter per token, and a backward pass costs about two FLOP per token for each parameter through which the gradient must be propagated plus two FLOP per token for each parameter whose gradient is computed. For an update that trains $N_{\text{eff}}(\Delta_k)$ parameters over D_k tokens, and whose back-propagation must traverse $p_b(\Delta_k) \le p$ parameters to reach the earliest trainable parameter, the cost is modeled as

$$c(\Delta_k) = \kappa\, D_k[\,2p + 2p_b(\Delta_k) + 2N_{\text{eff}}(\Delta_k)\,], \tag{5}$$

where κ converts floating-point operations to a monetary or opportunity cost. Full retraining sets $N_{\text{eff}} = p_b = p$, recovering the familiar $6pD_k$. Parameter-efficient fine-tuning and targeted knowledge editing set $N_{\text{eff}} \ll p$, but still pay the forward pass through all p parameters and a backward pass through p_b of them, so their cost per token is at least one third of that of full fine-tuning. A simpler *trainable-parameter accounting*,

$$c_{\text{tp}}(\Delta_k) = 6\kappa\, N_{\text{eff}}(\Delta_k)\, D_k, \tag{6}$$

coincides with (5) for full fine-tuning but understates the cost of parameter-efficient mechanisms; it is a lower bound that isolates the part of the cost that differs between mechanisms. Retrieval-augmentation events instead incur a much smaller, largely θ-independent indexing cost c_{RAG} that scales with corpus size rather than model size.

3.6. The Maintenance Objective

Collecting the three quantities above, the long-run maintenance problem is to choose a (possibly randomized, possibly history-dependent) sequence of maintenance times $\{t_k\}$, displacements $\{\Delta_k\}$, and retrieval-corpus updates $\{\mathcal{C}_{t_k}\}$ to minimize the long-run average cost rate

$$J = \limsup_{T\to\infty} \frac{1}{T}\,\mathbb{E}\left[\int_0^T \alpha\, X(t)\, dt + \sum_{k:\, t_k \le T} \Big(c(\Delta_k) + \beta\, G(\Delta_k)\Big)\right], \tag{7}$$

where α converts degradation into an operational loss rate (for example, lost users, compliance exposure, or downstream error propagation) and β is a one-off cost per unit of forgetting incurred at the moment of the update (for example, regression triage, re-certification, or user-visible behavioral regressions). Forgetting is thus charged twice by design: once as a one-off event cost through β, and once as a flow cost through α for as long as the residual damage remains in X. Setting $\beta = 0$ keeps only the flow cost. Equation (7) is deliberately structured so that its terms map one-to-one onto Section 4 (the $S(t)$ dynamics), Section 5 (the $G(\Delta_k)$ dynamics), and the cost model above; Section 6 shows how to jointly optimize over $\{t_k, \Delta_k\}$ under this objective using dynamic programming.

3.7. Assumptions

The following assumptions are maintained throughout the paper. Their scope and the consequences of relaxing them are discussed in Section 10.

- **A1** (Scalar, additive accuracy). $A(\theta, w)$ is a scalar summary of accuracy, and staleness and forgetting are commensurable and additive in X; in practice a deployed model is evaluated along many task-specific axes that can drift at different rates.
- **A2** (Exogenous world). The world process w_t is exogenous: the model's own outputs do not feed back into the distribution of tasks or facts it will later be asked about.
- **A3** (Discrete maintenance). Maintenance actions are discrete events; continuous, always-on adaptation is a limiting case as the minimum inter-event time shrinks to zero.
- **A4** (Independent marks). Drift magnitudes M_i are i.i.d. with distribution F_M and finite mean μ_M, independent of the arrival process $\{\tau_i\}$.
- **A5** (Local optimality). Immediately before an update, $\theta_{t_k^-}$ is (approximately) a local minimum of the loss on the old world distribution, so the loss admits a second-order expansion in the displacement with a diagonal Fisher information matrix F.
- **A6** (Monotone structure). In the MDP of Section 6, for each fixed drift regime: (i) the tail probabilities $q(k \mid x, a)$ of the next degradation state are non-decreasing in the current state x for each fixed action; (ii) the action set is totally ordered by non-decreasing

effectiveness $\rho(a)$ and non-decreasing cost $c(a)$; (iii) $q(k \mid x, a)$ is submodular in (x, a) for every k. In addition, the MDP is unichain.

4. A Stochastic Model of Knowledge Staleness

4.1. Drift as a Marked Point Process

We model the arrival of *drift events*—discrete moments at which the world state w_t shifts enough to be behaviorally consequential (a new fact supersedes an old one, a policy changes, a new class of user request emerges)—as a marked point process $\{(\tau_i, M_i)\}_{i \geq 1}$, where τ_i are arrival times and $M_i \geq 0$ are drift magnitudes satisfying Assumption A4. In the simplest case the arrival process is Poisson with rate λ, giving a compound Poisson staleness accumulation process; more generally, as emphasized by Beytur et al. [18] and consistent with the taxonomy of sudden, gradual, and incremental drift in Lu et al. [5], the inter-arrival times may follow a general distribution F_τ with hazard rate $h_\tau(\cdot)$, allowing drift seasons to exhibit increasing or decreasing mean residual life.

Between drift events, staleness accumulates continuously through ordinary exposure to a slowly moving world (vocabulary drift, slow factual turnover), modeled as a deterministic leak term, and jumps discretely at each drift event. The resulting staleness process obeys

$$S(t) = \eta\,(t - t_k) + \sum_{i:\,\tau_i \in (t_k, t]} M_i, \qquad t \in (t_k, t_{k+1}], \tag{8}$$

with $S(t_k^+) = 0$ immediately after the k-th maintenance event, $\eta \geq 0$ a background drift-leak rate, and the sum ranging over drift events since the last maintenance. This is a shock-and-wear degradation model of the kind used for physical systems [3], with the wear component η and the shock component given by the marked point process. By Wald's identity and Assumption A4, in expectation,

$$\mathbb{E}[S(t)] = \eta\,(t - t_k) + \Lambda(t - t_k)\,\mu_M, \tag{9}$$

where $\Lambda(v) = \mathbb{E}[N(v)]$ is the renewal function of the drift-arrival process over an interval of length v. In the Poisson special case $\Lambda(v) = \lambda v$, recovering the familiar result that expected staleness grows affinely in time since last maintenance, with slope $\gamma \equiv \eta + \lambda\mu_M$—a single effective decay rate that a practitioner could in principle estimate from monitoring data, exactly as advocated in drift-detection practice [5]. Since staleness is bounded (Section 3.2), this affine form is a local approximation; in a finite world of facts it gives way to saturation as drift events begin to revisit facts that are already stale.

4.2. A Renewal-Reward Formulation of the Retraining Decision

Consider first the restricted policy class in which maintenance is always full retraining (so $G(\Delta_k) \equiv 0$ by construction, since the retrained model is refit to the current world, and $X = S$) triggered whenever accumulated staleness first reaches a fixed threshold s. This is a control-limit policy on the degradation index in the sense of CBM [4], and it defines a renewal process on the staleness state: each maintenance event resets $S(t)$ to zero and the system regenerates probabilistically. We analyze it under a *mean-path (fluid) approximation* in which the cycle length is the time at which expected staleness reaches the threshold, $T(s) = \inf\{v \geq 0 : \mathbb{E}[S(v)] \geq s\}$. The exact first-passage time of the jump process (8) differs from $T(s)$ because of the overshoot at the crossing jump; the approximation is accurate when individual drift magnitudes are small relative to s. Let c_r denote the retraining cost

from Section 3. By the renewal-reward theorem [19], the long-run average cost rate under this policy is

$$J(s) = \frac{c_r + \alpha \int_0^{T(s)} \mathbb{E}[S(v)]\, dv}{T(s)}. \tag{10}$$

Substituting the affine staleness growth $\mathbb{E}[S(v)] = \gamma v$ and noting $T(s) = s/\gamma$, the integral evaluates in closed form to $\alpha\gamma T(s)^2/2 = \alpha s^2/(2\gamma)$, giving

$$J(s) = \frac{c_r\,\gamma}{s} + \frac{\alpha\, s}{2}. \tag{11}$$

Minimizing over s by setting $dJ/ds = 0$ yields the classical square-root economic-order-quantity form familiar from inventory and reliability theory,

$$s^\star = \sqrt{\frac{2\,c_r\,\gamma}{\alpha}}, \qquad T^\star = \frac{s^\star}{\gamma} = \sqrt{\frac{2\,c_r}{\gamma\,\alpha}}, \qquad J^\star = \sqrt{2\,c_r\,\gamma\,\alpha}. \tag{12}$$

Two qualitative predictions follow immediately. First, the optimal time between retraining events, $T^\star$, shrinks as the drift intensity γ grows, while the staleness level tolerated before retraining, $s^\star$, grows; faster drift calls for more frequent maintenance in calendar time even though each cycle ends at a higher staleness level. This prediction has not yet been tested directly: Williams et al. [16] varied the retraining cadence rather than the drift intensity. Second, $T^\star$ grows only as the square root of the retraining cost c_r, so an order-of-magnitude increase in the cost of retraining a larger model (as implied by the cost model of Section 3) only lengthens the optimal retraining interval by a factor of about three, not by an order of magnitude. This diminishing-returns relationship is consistent with the observation in [16] that monthly retraining was non-inferior to weekly retraining at a quarter of the computational burden, and it provides a mathematical explanation for why naive "just retrain less because it is expensive" heuristics can be badly miscalibrated.

4.3. Beyond the Poisson Case: Hazard-Rate-Dependent Policies

When drift arrivals are not Poisson, the renewal function $\Lambda(v)$ is no longer linear, and the conditional intensity of drift over the coming interval depends on the elapsed time u since the last drift event—information that a policy monitoring only the staleness level ignores. Maintenance cannot pre-empt drift that has not yet occurred; elapsed time matters because it changes the rate at which staleness is expected to accumulate after the next maintenance event, and therefore, through the square-root relation (12), the economically appropriate cycle length. The threshold policy of Section 4.2 remains well defined but is no longer necessarily optimal among history-dependent policies, and a time-based or hybrid trigger may do better.

Beytur et al. [18] show, for a budget-constrained training problem with sudden concept changes, that the structure of the optimal policy depends on the aging property of the concept-duration distribution: they derive optimal policies when durations have decreasing mean residual life (DMRL) and prove that intuitive heuristics are suboptimal when durations have increasing mean residual life (IMRL). Carrying the aging notion over to the present setting yields two hypotheses, which we state as such rather than as theorems. Under DMRL, a long quiet interval signals that the next drift event is expected sooner, which raises the near-term effective drift rate and, by (12), calls for a shorter calendar interval to the next maintenance event. Under IMRL, a long quiet interval signals a longer expected wait for the next drift event, and the opposite adjustment is indicated; a heuristic that treats every quiet spell as a reason to maintain sooner is then miscalibrated. Whether these adjustments yield material cost savings in a given deployment is an empirical question that

the regime-dependent MDP of Section 6 answers numerically for a fitted drift model. The two regimes are distinguishable from monitoring data, via estimation of the hazard rate of drift-detector alarms, and they imply opposite prescriptions for how a scheduler should react to an unusually long quiet interval—a distinction invisible to policies that monitor only accumulated staleness.

4.4. Estimating the Decay Parameters from Monitoring Data

The parameters η, λ, and μ_M (or, more generally, the renewal function $\Lambda(\cdot)$) are not directly observed but can be estimated from a monitoring pipeline that periodically evaluates the deployed model against held-out, freshly collected evaluation data—the same operational infrastructure already recommended in the concept-drift literature [5] and increasingly standard in MLOps practice for classical models [6,27]. Because $\mathbb{E}[S(v)]$ is, under the model of (9), affine in v between maintenance events, γ can be recovered by ordinary least squares (OLS) regression of measured staleness on elapsed time since the last maintenance event, using data pooled across multiple maintenance cycles. A nonzero fitted intercept reflects residual error of the freshly maintained model and does not affect the slope. Departures from affinity in this regression are themselves diagnostic, either of non-Poisson drift arrivals, which motivates the hazard-rate-based refinement of Section 4.3, or of saturation in a finite world of facts. Section 8.4 carries out exactly this regression on a public pretrained model.

5. Fisher-Information-Constrained Update Dynamics: A Model of Catastrophic Forgetting

5.1. Forgetting as a Function of Displacement Geometry

Following the Bayesian derivation underlying EWC [9] and Assumption A5, we approximate the pre-update loss surface $L(\theta_{t_k^-} + \Delta)$, restricted to the old world distribution $w_{t_k^-}$, by its second-order Taylor expansion around $\theta_{t_k^-}$:

$$L\left(\theta_{t_k^-} + \Delta\right) \approx L\left(\theta_{t_k^-}\right) + \tfrac{1}{2}\,\Delta^\top F\,\Delta, \tag{13}$$

where F is the (diagonal-approximated) Fisher information matrix of the pre-update model with respect to the old task distribution, and the first-order term vanishes because $\theta_{t_k^-}$ is (approximately) a local minimum of the pre-update loss. Under this approximation, and assuming that the accuracy loss on the old world is, to leading order, proportional to the increase in its loss (the proportionality constant is absorbed into the units of G), the forgetting functional defined in (3) is a quadratic form in the displacement:

$$G(\Delta) \approx \frac{1}{2}\,\Delta^\top F\,\Delta = \frac{1}{2}\sum_{i=1}^{p} F_i\,\Delta_i^2. \tag{14}$$

This is precisely the penalty term added to the training objective under EWC and its variants [9]; the contribution of this paper is to treat $G(\Delta)$ not merely as a regularizer to be added to a loss function during optimization, but as an explicit constraint set entering the maintenance cost objective of Section 3, so that the amount of forgetting an organization is willing to purchase in a given update becomes a tunable, auditable quantity rather than an implicit consequence of a regularization hyperparameter.

5.2. The Constrained Update Problem

Given a target reduction in staleness $\bar{S}$ that a maintenance action is intended to achieve, and given the quadratic forgetting approximation above, the update-selection problem at maintenance time t_k is

$$\Delta_k^\star = \operatorname*{argmin}_{\Delta} \; c(\Delta) \quad \text{s.t.} \quad \Delta^\top F \Delta \le 2G_{\max}, \quad P(\Delta) \ge \bar{S}, \tag{15}$$

where $G_{\max}$ is an organization's tolerance for forgetting expressed in the same accuracy units as staleness, $P(\Delta)$ is the staleness reduction (*plasticity*) delivered by Δ on the new data, and $c(\Delta)$ is the cost functional of Section 3. By (5), the mechanism-dependent part of $c(\Delta)$ is $2p_b + 2N_{\text{eff}}$ per token, which grows with the number of parameters touched—that is, with $\|\Delta\|_0$ or a soft relaxation thereof—and with how deep in the network they lie. This problem therefore exhibits a direct trade-off between the sparsity and locality of the update, its cost, and its forgetting footprint, with the Fisher information matrix F acting as an anisotropic metric that makes some directions in parameter space far more expensive (in forgetting terms) to move along than others. This formalizes the widely observed empirical fact that knowledge-editing and parameter-efficient fine-tuning methods—which restrict Δ to a low-dimensional or sparse subspace by construction [12,13]—*can* achieve much lower forgetting than unconstrained full-parameter fine-tuning for a comparable gain in target-task performance. The qualification "can" matters: a low-rank or sparse subspace bounds $\|\Delta\|_0$, not $\Delta^\top F \Delta$, and Section 8.4 exhibits a parameter-efficient update that violates the forgetting constraint.

5.3. The Stability–Plasticity Pareto Frontier

Define the plasticity achieved by an update as the staleness reduction it delivers, $P(\Delta) \equiv S(t_k^-) - S(t_k^+; \Delta)$, and its stability cost as the forgetting $G(\Delta)$ from (14). For a fixed cost budget $c(\Delta) \le \bar{c}$, tracing out the efficient frontier of the bi-objective problem $\max P(\Delta)$, $\min G(\Delta)$ over admissible Δ produces a Pareto frontier whose shape is governed by the alignment between the gradient direction that reduces staleness on the new data and the principal eigenvectors of F. When these directions are nearly orthogonal—that is, when the knowledge needed to address the new drift event occupies parameter directions to which the old task is insensitive—the frontier is nearly flat and large plasticity gains can be purchased at negligible stability cost; this is the regime knowledge-editing methods are designed to exploit by explicitly localizing new facts to under-used parameter subspaces [12, 13]. When the two directions are nearly parallel—for instance, when a broad alignment update of the kind studied by Ouyang et al. [26] necessarily moves parameters that are heavily used across almost the entire pre-training distribution—the frontier is steep, and every unit of new-behavior plasticity is bought at substantial cost to old-behavior stability. This is exactly the alignment tax documented empirically in [26] and mitigated there by blending the reinforcement-learning update with the original pre-training loss, a strategy that biases Δ toward directions with a smaller F-weighted norm.

5.4. Incorporating Forgetting into the Renewal-Reward Objective

A nonzero forgetting event $G(\Delta)$ leaves the model with residual damage at the moment of maintenance, so the degradation index (4) restarts at $X(t_k^+) = G(\Delta)$ rather than at zero. This is the imperfect-repair generalization of the perfect-repair renewal model [1]. Consider a policy that applies the same mechanism Δ whenever X reaches a threshold $s > G(\Delta)$. Under the mean-path approximation of Section 4.2, a cycle lasts $T(s, \Delta) = \big(s - G(\Delta)\big)/\gamma$,

and the expected degradation integrated over the cycle is $G(\Delta)\,T + \gamma T^2/2$. Charging the one-off cost $c(\Delta) + \beta G(\Delta)$ once per cycle, as in (7), yields the long-run cost rate

$$J(s,\Delta) = \frac{1}{T(s,\Delta)}\left[c(\Delta) + \beta\, G(\Delta) + \alpha\left(\frac{\big(s - G(\Delta)\big)^2}{2\gamma} + G(\Delta)\, T(s,\Delta)\right)\right]. \tag{16}$$

Writing $v = s - G(\Delta)$, (16) becomes $J = (c + \beta G)\gamma/v + \alpha v/2 + \alpha G$, which is minimized at

$$\begin{aligned} s^\star(\Delta) &= G(\Delta) + \sqrt{\frac{2\gamma\big(c(\Delta) + \beta G(\Delta)\big)}{\alpha}}, \\ T^\star(\Delta) &= \sqrt{\frac{2\big(c(\Delta) + \beta G(\Delta)\big)}{\alpha\,\gamma}}, \\ J^\star(\Delta) &= \sqrt{2\gamma\alpha\big(c(\Delta) + \beta G(\Delta)\big)} + \alpha\, G(\Delta). \end{aligned} \tag{17}$$

Three consequences follow. First, forgetting raises the optimal cost rate through two channels: it inflates the effective one-off cost from c to $c + \beta G$, and it adds a permanent flow cost αG that no choice of cadence can remove. Second, holding c fixed, a mechanism with lower forgetting has a smaller effective one-off cost, so both the optimal threshold $s^\star(\Delta)$ and the optimal interval $T^\star(\Delta)$ decrease: cheaper, safer maintenance events should be carried out *more* often, not less often. Third, when mechanisms differ in both c and G, they should be compared by $J^\star(\Delta)$ in (17), not by c or G alone; a cheap mechanism with a large G can be worse than an expensive one with $G \approx 0$ because of the permanent term αG. Minimization of (17) over the admissible mechanisms—equivalently, over points on the Pareto frontier of Section 5.3—is precisely the integration promised in Section 3.

6. A Markov Decision Process Formulation of Maintenance Scheduling

6.1. State, Action, and Transition Structure

To move beyond the single-threshold policies of Sections 4 and 5 and allow for a menu of maintenance mechanisms with distinct cost, forgetting, and effectiveness profiles, we discretize time into decision epochs $t = 0, 1, 2, \ldots$ and define the state at each epoch as $x_t = (X_t, d_t) \in \mathcal{X} \times \mathcal{D}$, where X_t is the (discretized) degradation index from Section 3.4 and d_t is an estimate of the current drift-hazard regime from Section 4.3 (for instance, a coarse indicator of whether the elapsed-time-since-last-drift-event process is currently in a DMRL- or IMRL-consistent phase, updated via a filtering procedure applied to drift-detector alarms). The regime d_t is assumed to evolve as an exogenous Markov chain, independent of X_t and of the actions. At each epoch the maintainer chooses a parametric maintenance action

$$a_t \in \mathcal{A} = \{\text{no-op},\ \text{knowledge-edit},\ \text{PEFT},\ \text{full retrain}\}, \tag{18}$$

each associated with a cost $c(a)$ (Section 3), a forgetting increment $G(a)$ (Section 5), and a degradation-reduction fraction $\rho(a) \in [0, 1]$ such that

$$X_{t+1} = \big(1 - \rho(a_t)\big)\, X_t + G(a_t) + \xi_{t+1}, \tag{19}$$

where $\xi_{t+1} \geq 0$ is the (random) staleness increment accrued during the epoch under the drift model of Section 4, with a distribution determined by d_t. Retrieval augmentation is deliberately not an element of $\mathcal{A}$: it does not reduce the degradation already accumulated in θ but lowers the rate at which new staleness registers, so it enters the MDP by changing the distribution of ξ_{t+1} (its mean falls from γ to $\gamma - \bar{\phi}$ per unit time) at an upkeep cost c_{RAG}

per unit time. Whether to maintain the retrieval store is then an outer decision, taken by comparing the optimal average cost of the MDP with and without retrieval (Section 7.2).

6.2. The Bellman Equation

The per-epoch cost is $r(x_t, a_t) = \alpha X_t + \beta G(a_t) + c(a_t)$, and the long-run average-cost optimality equation (the Bellman equation for this MDP, following standard dynamic-programming theory [20,21]) is

$$h(x) + g = \min_{a \in \mathcal{A}} \{ r(x, a) + \mathbb{E}[h(x') \mid x, a] \}, \tag{20}$$

where g is the optimal long-run average cost rate and $h(\cdot)$ is a relative value function unique up to an additive constant. Because $\mathcal{A}$ is a small, finite set and $\mathcal{X} \times \mathcal{D}$ can be discretized to any desired resolution, (20) is directly solvable by standard relative value iteration or policy iteration for any specific numerical instantiation of the cost, forgetting, and drift parameters, and does not require closed-form solution in general; the purpose of this section is instead to establish qualitative structural properties of the optimal policy that hold across a broad range of parameterizations.

6.3. Monotonicity and the Threshold-Policy Result

Proposition 1 (Monotone, threshold-structured policy)**.** *Suppose Assumption A6 holds, that is, for each fixed drift regime d: (i) the tail probabilities $q(k \mid X, a) = \Pr(X_{t+1} \geq k \mid X_t = X, a_t = a)$ are non-decreasing in X for every k and a; (ii) the action set $\mathcal{A}$ is totally ordered, no-op $\prec$ knowledge-edit $\prec$ PEFT $\prec$ full retrain, with $\rho(a)$ and $c(a)$ non-decreasing along the order; (iii) $q(k \mid X, a)$ is submodular in (X, a) for every k, that is, $q(k \mid X, a') - q(k \mid X, a)$ is non-increasing in X whenever $a' \succ a$; and the MDP is unichain. Then the relative value function $h(X, d)$ is non-decreasing in X for each d, and there is an average-cost optimal policy $a^\star(X, d)$ that is non-decreasing in X: there exist thresholds $0 \leq X_1(d) \leq X_2(d) \leq \cdots \leq X_{|\mathcal{A}|-1}(d)$ such that the optimal action escalates through the ordered action set as X crosses each threshold in turn.*

Proof. Consider first the δ-discounted problem, $\delta \in (0, 1)$, and its value iterates $V_{n+1}(X, d) = \min_a Q_n(X, d, a)$ with $Q_n(X, d, a) = r(X, a) + \delta\, \mathbb{E}[V_n(X', d') \mid X, d, a]$ and $V_0 \equiv 0$. Suppose $V_n(\cdot, d')$ is non-decreasing for every d'. Because the regime evolves independently of X and a, the expectation can be written, for each d', as $V_n(0, d') + \sum_{k \geq 1} [V_n(k, d') - V_n(k-1, d')]\, q(k \mid X, a)$, a non-negative combination of the tail probabilities. By (i), this expectation is non-decreasing in X; since $r(X, a)$ is non-decreasing in X ($\alpha > 0$), Q_n and hence $V_{n+1} = \min_a Q_n$ are non-decreasing in X. By (iii) and the non-negativity of the increments of V_n, the expectation is submodular in (X, a); the one-step cost $r(X, a) = \alpha X + \beta G(a) + c(a)$ is separable in (X, a) and therefore modular. Hence $Q_n(\cdot, d, \cdot)$ is submodular in (X, a), and by Topkis's monotonicity theorem [28] the largest minimizer of $Q_n(X, d, \cdot)$ is non-decreasing in X. By induction these properties hold for every n, and they pass to the limit $V_\delta = \lim_n V_n$, which exists because the state and action sets are finite; so for every δ there is a monotone δ-discount-optimal policy [21]. Because there are finitely many deterministic stationary policies, one monotone policy is discount-optimal along a sequence $\delta_m \to 1$; under the unichain assumption it is average-cost optimal, and $h = \lim_m [V_{\delta_m} - V_{\delta_m}(x_{\text{ref}})]$ inherits the monotonicity of V_{δ_m} [21]. A monotone map from an ordered state set to a totally ordered finite action set is characterized by the thresholds stated. □

Two remarks clarify the scope of the conditions. Condition (i) holds automatically for the transition (19), because $1 - \rho(a) \geq 0$. Condition (iii) is not automatic: under (19), $q(k \mid X, a) = \Pr(\xi \geq k - (1 - \rho(a))X - G(a))$, and submodularity depends on the tail

shape of ξ and on the forgetting increments $G(a)$. It holds without further conditions for replacement-type actions: if $\rho(a') = 1$, then $q(k \mid X, a')$ does not depend on X while $q(k \mid X, a)$ is non-decreasing in X, so the difference is non-increasing. In general, (iii) can be verified numerically on the discretized kernel. The result does not assert that h is convex; convexity is not needed for the argument and is not preserved in general by the minimization over a discrete action set.

This proposition generalizes the single-threshold result of Section 4.2 (recovered as the special case $|\mathcal{A}| = 2$, a choice only between no-op and full retrain, with $G \equiv 0$ so that forgetting plays no role) to the realistic case of a graduated menu of maintenance mechanisms. It formally justifies practices already emerging informally in MLOps pipelines—escalating from a targeted knowledge edit, to a broader parameter-efficient fine-tune, and only as a last resort to full retraining, while a retrieval store is maintained alongside as a separate compensating input—as the optimal qualitative shape of a maintenance policy under the stated conditions, rather than a merely convenient engineering heuristic. It is the multi-action analogue of the control-limit policies that are optimal for monotone degradation processes in CBM [4].

The thresholds $X_j(d)$ depend on the drift regime d, but Proposition 1 does not determine the direction of that dependence, and the closed form of Section 4.2 shows why intuition can mislead here: in (12), a higher drift intensity γ *raises* the optimal staleness threshold $s^\star$ while *shortening* the optimal calendar interval $T^\star$. In a high-hazard regime the policy therefore maintains more often in calendar time, but it need not do so at a lower degradation level. The direction of the threshold shift for a fitted model is obtained numerically by the procedure of Section 6.4, which generalizes to a graduated action set the question that Beytur et al. [18] study for the train-or-not case.

6.4. Numerical Solution via Relative Value Iteration

For any specific parameterization of the cost, forgetting, and drift-transition functions introduced above, (20) can be solved numerically without appeal to Proposition 1 at all, which provides both a practical route to deployment and an independent check on the proposition's predictions. Relative value iteration proceeds by fixing a reference state x_{ref}, initializing $h_0(x) \equiv 0$, and iterating

$$\begin{aligned} h_{n+1}(x) = {} & \min_{a \in \mathcal{A}} \big\{ r(x,a) + \mathbb{E}\big[h_n(x') \mid x, a\big] \big\} \\ & - \min_{a \in \mathcal{A}} \big\{ r(x_{\mathrm{ref}},a) + \mathbb{E}\big[h_n(x') \mid x_{\mathrm{ref}}, a\big] \big\} \end{aligned} \tag{21}$$

until h_{n+1} and h_n agree to within a specified tolerance, at which point the minimizing action at each state is read off as the (numerically) optimal policy and the subtracted term converges to the optimal average cost rate $g^\star$; if the chain is periodic, the standard aperiodicity transformation is applied first [21]. Because the state space $\mathcal{X} \times \mathcal{D}$ used in this paper is low-dimensional by design (a scalar discretized degradation level crossed with a small number of drift-hazard regimes), this computation is inexpensive relative to the cost of even a single knowledge-editing operation on a deployed model, and can in principle be re-solved on a rolling basis as the underlying drift parameters are re-estimated from monitoring data as described in Section 4.4, yielding an adaptive rather than purely static maintenance policy. When the conditions of Proposition 1 hold, the numerically obtained policy can be chosen to exhibit the threshold structure derived analytically; when they do not, relative value iteration remains valid and simply returns whatever (possibly non-monotone) policy is in fact optimal, making it a useful diagnostic for detecting violations of the proposition's assumptions in a fitted, real-world instantiation of the model.

6.5. Discussion of the Structural Result

Two caveats qualify Proposition 1's practical reach. First, the forgetting increment in (19) is taken to depend on the action only. If it also depends on the current state—for instance, if attempting an aggressive fine-tune from an already very stale starting point induces disproportionately severe forgetting because the optimizer must travel further in parameter space, so that $G = G(a, X)$—then both condition (i) and condition (iii) can fail. The Pareto-frontier analysis of Section 5.3 suggests this is a real possibility whenever the required displacement direction rotates substantially as staleness accumulates. Second, condition (iii)'s submodularity encodes a specific, economically plausible but not universally guaranteed belief that a more aggressive action removes more of the probability of reaching high degradation levels when the current degradation is high; violations would produce a genuinely non-monotone optimal policy, which is a testable prediction of the model rather than an assumption to be taken on faith. Condition (ii) is likewise a property to be verified per mechanism, and Section 8.4 exhibits a measured violation of it.

7. Retrieval Augmentation as a Feedback Compensator

7.1. A Control-Theoretic Reading of RAG

The maintenance mechanisms considered in Sections 5 and 6—knowledge editing, parameter-efficient fine-tuning, full retraining—all act by displacing θ_t, and therefore all incur some forgetting risk $G(\Delta_k) \geq 0$ under the model of Section 5. RAG [14] is structurally different: it reduces *observed* staleness by conditioning generation at inference time on passages drawn from an external corpus $\mathcal{C}_t$ that can be updated independently and continuously, without touching θ_t at all. In the language of (1), a well-functioning RAG system effectively replaces the freshness functional $A(\theta_t, w_t)$ with an augmented functional $A(\theta_t, \mathcal{C}_t, w_t) \geq A(\theta_t, w_t)$ whenever $\mathcal{C}_t$ contains information relevant to closing the gap between w_{t_k} and w_t, while leaving θ_t, and hence $G(\cdot)$, entirely unaffected.

This is naturally read as feedback control: if we regard staleness $S(t)$ as an error signal, a parametric update is analogous to adjusting the plant itself (changing θ, with attendant risk of destabilizing previously tuned behavior), whereas a retrieval-corpus refresh is analogous to adjusting a compensating input that corrects the observed error without modifying the plant's internal dynamics at all. Formally, define an *effective staleness* under retrieval augmentation

$$\tilde{S}(t) = S(t) - \phi(\mathcal{C}_t, w_t), \tag{22}$$

where $\phi(\cdot) \geq 0$ is the staleness reduction attributable to retrieval, increasing in the recency and coverage of $\mathcal{C}_t$ relative to w_t and bounded above by the *retrievability* of the relevant knowledge—that is, by how far the drift in question (a new fact, easily indexed, versus a change in required reasoning style or task format, which retrieval cannot easily supply) is amenable to correction via retrieved text at all.

7.2. Optimal Blending of Retrieval and Parametric Maintenance

Suppose the retrieval store is kept current at an upkeep cost c_{RAG} per unit time and absorbs drift at an average rate $\bar{\phi} < \gamma$, so that effective staleness grows at rate $\gamma - \bar{\phi}$

between parametric maintenance events. Substituting $\tilde{S}(t)$ for $S(t)$ in the renewal-reward analysis of Section 4.2 gives $J_{\mathrm{RAG}}(s) = c_r(\gamma - \bar{\phi})/s + \alpha s/2 + c_{\mathrm{RAG}}$, whose minimizer is

$$\begin{aligned} s^{\star}_{\mathrm{RAG}} &= \sqrt{\frac{2\,c_r\,(\gamma - \bar{\phi})}{\alpha}}, \\ T^{\star}_{\mathrm{RAG}} &= \sqrt{\frac{2\,c_r}{\alpha\,(\gamma - \bar{\phi})}} = \frac{T^{\star}}{\sqrt{1 - \bar{\phi}/\gamma}}, \\ J^{\star}_{\mathrm{RAG}} &= \sqrt{2\,c_r\,(\gamma - \bar{\phi})\,\alpha} + c_{\mathrm{RAG}}. \end{aligned} \tag{23}$$

Maintaining the retrieval store thus *lowers* the optimal staleness threshold relative to the no-RAG optimum $s^{\star}$ of (12), by the factor $\sqrt{1 - \bar{\phi}/\gamma}$, while *lengthening* the optimal interval between parametric maintenance events by the factor $1/\sqrt{1 - \bar{\phi}/\gamma}$: the model is retrained less often, and because staleness accumulates more slowly, it is retrained at a lower staleness level. Comparing (23) with (12) gives the condition under which blending dominates parametric maintenance alone,

$$c_{\mathrm{RAG}} < \sqrt{2\,c_r\,\gamma\,\alpha}\left(1 - \sqrt{1 - \bar{\phi}/\gamma}\right). \tag{24}$$

Retrieval alone, with no parametric maintenance, is never optimal when $\bar{\phi} < \gamma$, because effective staleness then grows without bound under the affine model and the average cost diverges. Blending therefore dominates both pure strategies exactly when (24) holds. This formalizes the widely held but rarely quantified engineering intuition that a well-maintained retrieval system is a substitute—over the portion of drift it can absorb—for more frequent and more forgetting-prone parametric updates, while making explicit that the substitution is only partial and bounded by $\bar{\phi}$: forms of drift that are not amenable to retrieval-based correction (shifts in required reasoning patterns, output format conventions, or safety norms that are not simply facts to be looked up) still accumulate at their full rate against the parametric model and must eventually be addressed via the mechanisms of Sections 5 and 6. The case study of Section 8.4 measures $\bar{\phi}/\gamma = 0.61$ for a retrieval store covering 60% of the drifting facts.

7.3. Failure Modes Not Captured by Staleness Reduction Alone

It is worth noting explicitly that $\phi(\cdot)$ can be negative in effect even when nominally positive in design if the retrieved evidence is irrelevant or contradictory, a failure mode connected in the broader literature to the general problem of hallucination in generated text [29]; the reduction in measured staleness attributable to RAG is therefore best treated, for maintenance-planning purposes, as an empirically monitored quantity subject to the same estimation procedure outlined in Section 4.4, rather than as a fixed design parameter that can be assumed correct once a retrieval system has been deployed.

8. Numerical Results and Findings

8.1. Purpose and Setup

Sections 8.2 and 8.3 illustrate the qualitative behavior implied by Sections 4–7 using stylized parameter values chosen for interpretability rather than calibration to any specific deployed system; the purpose is to make the comparative statics of the model concrete, not to produce a validated forecast. Section 8.4 then instantiates the measured inputs of the framework on a public pretrained model. We instantiate the single-threshold model of Section 4.2 with a background leak rate $\eta = 0.01$ staleness units per day, a drift-event rate λ swept over $\{0.05, 0.1, 0.2, 0.4\}$ events per day, a mean drift magnitude $\mu_M = 1$ (so that a typical drift event raises staleness by one unit at the moment it occurs), and an

operational-loss coefficient $\alpha = 1$ cost unit per staleness unit per day. Retraining cost c_r is swept over three orders of magnitude, $\{10^2, 10^4, 10^6\}$ cost units, to represent a small fine-tune, a mid-size continual-pretraining run, and a large-scale full retrain respectively, loosely consistent with the compute-cost scaling of Section 3 applied to models differing by roughly two orders of magnitude in parameter count. Because staleness units and cost units are stylized, only ratios between entries of Table 3 are meaningful.

8.2. Qualitative Predictions

Table 3 reports the optimal cycle length and cost rate from the closed form (12) over this grid, and three qualitative patterns emerge that are worth stating explicitly as testable implications of the model.

Table 3. Optimal cycle length $T^\star$ (days) and cost rate $J^\star$ from (12), with $\eta = 0.01$, $\mu_M = 1$, $\alpha = 1$.

λ (per Day)	γ	$T^\star$, $c_r = 10^2$	$T^\star$, $c_r = 10^4$	$T^\star$, $c_r = 10^6$	$J^\star$, $c_r = 10^2$	$J^\star$, $c_r = 10^4$	$J^\star$, $c_r = 10^6$
0.05	0.06	57.7	577.4	5773.5	3.46	34.64	346.41
0.10	0.11	42.6	426.4	4264.0	4.69	46.90	469.04
0.20	0.21	30.9	308.6	3086.1	6.48	64.81	648.07
0.40	0.41	22.1	220.9	2208.6	9.06	90.55	905.54

First, holding c_r fixed, the optimal retraining interval $T^\star = \sqrt{2c_r/(\gamma\alpha)}$ falls by a factor of two whenever the drift intensity γ quadruples, so a fourfold acceleration in the rate at which the world changes around a deployed model justifies only a halving, not a quartering, of the maintenance interval—a much more conservative adjustment than a naive "match the drift rate" heuristic would suggest. Across the grid, γ rises by a factor of 6.8 and $T^\star$ falls by a factor of 2.6 in every column. Second, holding drift intensity fixed, moving from the smallest to the largest retraining-cost regime considered here (a factor of 10^4) lengthens the optimal interval by only a factor of 100, again reflecting the diminishing sensitivity of the optimum to cost that is structurally guaranteed by the square-root relationship, and consistent with the empirical observation in [16] that a fourfold reduction in retraining frequency (from weekly to monthly) was non-inferior while saving three quarters of the computation. Third, the forgetting-augmented optimum (17) shows that, for a fixed cost c, a mechanism with a lower forgetting increment $G(\Delta)$ lowers the optimal cost rate through both the one-off term and the permanent term αG, and at the same time shortens the optimal interval, because the effective per-event cost $c + \beta G$ falls. Investment in less-forgetting maintenance mechanisms (better knowledge localization, better regularization) therefore pays off both directly, by lowering the damage per event, and indirectly, by making it economical to maintain more often and so to run at a lower average staleness.

8.3. Predicted Sensitivity to the Drift-Hazard Regime

The two-regime extension of Section 4.3 can be instantiated with a Weibull inter-drift-event distribution. A shape parameter greater than one gives an increasing hazard rate and hence DMRL; a shape parameter less than one gives a decreasing hazard rate and hence IMRL; a shape parameter of one recovers the Poisson case. The hypotheses of Section 4.3 predict that under the DMRL parameterization, a policy that shortens the calendar interval to the next maintenance event as the time since the last drift event grows achieves a lower long-run cost rate than the memoryless threshold policy of Section 4.2, whereas under the IMRL parameterization the direction of the adjustment reverses and a policy that reacts to a long quiet spell by maintaining sooner performs worse than the memoryless policy. These predictions can be checked for any fitted drift model by solving the regime-dependent MDP with relative value iteration (Section 6.4). If confirmed, this would show that the

shape of the drift-arrival hazard function is not a second-order detail but a first-order determinant of which class of maintenance policy an organization should adopt, and that this determination is in principle estimable from historical drift-detector alarm data as outlined in Section 4.4.

8.4. Case Study with a Public Pretrained Model

To move beyond stylized parameters, we instantiate the measurement side of the framework—the staleness process of Section 4, the forgetting functional (3), and the cost model of Section 3.5—on a public pretrained model, EleutherAI Pythia-70M, using a synthetic world of facts that are guaranteed to be absent from its pre-training corpus. The world consists of 150 fictional companies, each described by four attributes: three dynamic attributes (chief executive, headquarters city, main product; 450 facts) that are subject to drift, and one static attribute (founding year; 150 facts) that never changes and serves as the retention suite for measuring forgetting. Each attribute has eight candidate values, and the model is scored as correct on a fact if the true value receives the highest summed token log-probability among the eight candidates given a fixed prompt. Staleness is the error rate on the dynamic suite in percentage points, and forgetting is the change in error rate on the static suite; one fact of the static suite corresponds to 0.67 percentage points. Drift events arrive according to a Poisson process with rate $\lambda = 3$ per day, each replacing one randomly chosen dynamic fact; 70% of drift events are assumed to be detected and logged, and 60% of the dynamic facts are indexed in an external retrieval store whose content is refreshed daily. The deployed model at $t = 0$ is obtained by fine-tuning the pretrained checkpoint until it answers at least 95% of both suites correctly, both with and without retrieved notes in the prompt. All results below are from a single random seed on a single GPU and should be read as a controlled measurement rather than as an estimate with error bars.

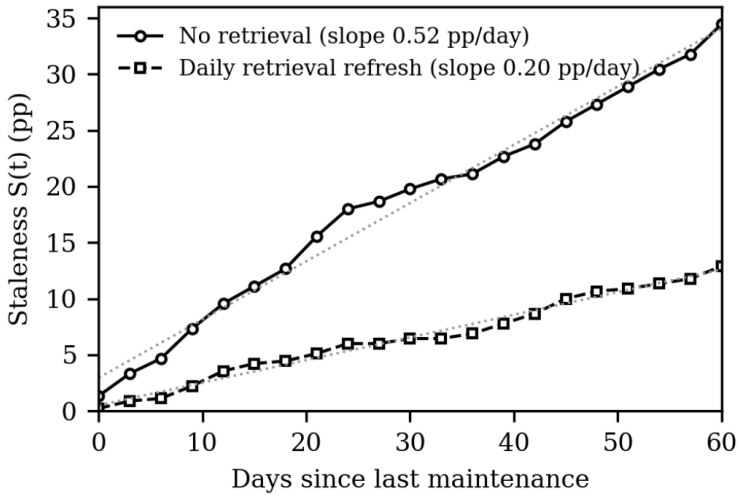


Figure 1. Error on the dynamic suite of the deployed Pythia-70M model over 60 days without parametric maintenance, measured every three days (single seed). Dotted lines are least-squares fits. The value at $t = 0$ is the residual error of the deployed model, which was trained to at least 95%, not 100%, accuracy; staleness $S(t)$ as defined in (2) is the curve minus this intercept, and the fitted slopes are unaffected. Retrieval over a corpus covering 60% of the drifting facts reduces the fitted slope from 0.52 to 0.20 percentage points per day, an absorbed share of 61%.

Figure 1 shows the growth of staleness over 60 days of unaddressed drift, measured every three days, with and without retrieval. Both trajectories are well described by the affine form of (9): OLS gives a slope of 0.519 percentage points per day without retrieval ($R^2 = 0.987$) and 0.202 percentage points per day with a daily retrieval refresh ($R^2 = 0.985$),

which is precisely the estimation procedure of Section 4.4. Over this horizon, 180 drift events are expected, and the expected fraction of distinct dynamic facts that have changed at least once is $1 - e^{-180/450} \approx 0.33$ rather than $180/450 = 0.40$; the affine fit is therefore a local approximation to a process that saturates on longer horizons (Section 3.2). The ratio of the two slopes yields a retrieval-absorbed drift share of $\bar{\phi}/\gamma = 1 - 0.202/0.519 = 0.611$, which matches the fraction of dynamic facts indexed in the retrieval store (0.60) to within the resolution of the experiment: retrieval absorbs the share of drift that it covers, and no more, in agreement with the boundedness of the compensator in Section 7.1. Substituting this share into (23) predicts that maintaining the retrieval store lengthens the economically optimal interval between parametric updates by a factor of $1/\sqrt{1 - 0.611} \approx 1.60$ at any fixed retraining cost, and lowers the optimal staleness threshold by the reciprocal factor, 0.62.

Table 4 reports the measured cost, error reduction, and forgetting of three parametric maintenance mechanisms, each applied to the same deployed model after 10 and 30 days of accumulated drift: a localized knowledge edit that fine-tunes only the multilayer perceptron (MLP) block of a single transformer layer on the facts of those entities for which a drift event was logged, a low-rank adaptation (LoRA, rank 16) trained on all dynamic facts without any replay of the static suite, and a full fine-tune of all parameters on all current facts. Costs are computed under the trainable-parameter accounting (6), as six times the number of trainable parameters times the number of tokens processed, and normalized so that a full fine-tune equals 1000 cost units.

Table 4. Measured cost, error reduction, and forgetting of three maintenance mechanisms on Pythia-70M.

Mechanism	**Trainable Subset**	**Cost, (6) (Units)**	**Error Before → After (pp)**	**Change in Retention Error (pp)**
Localized edit	MLP block of one layer (entities with logged drift)	14.0	17.4 → 13.8	−2.0
LoRA ($r = 16$, no replay)	Low-rank adapters, all layers	2.6	17.4 → 24.4	+11.0
Full fine-tune	All parameters	1000	17.4 → 5.7	−2.0

Values are averaged over interventions applied after 10 and 30 days of accumulated drift. Costs are normalized so that a full fine-tune equals 1000 units (about 5.1×10^{13} FLOP). Error is the sum of staleness on the dynamic suite and error on the static retention suite, i.e., the degradation index X of (4) plus the residual error of the deployed model. A change of −2.0 pp corresponds to three of the 150 retention facts and is within single-seed noise.

Two of the three mechanisms behave as the framework anticipates. The full fine-tune is the most effective, reducing total error from 17.4 to 5.7 percentage points, and by far the most expensive at 1000 cost units. The localized edit reduces total error from 17.4 to 13.8 percentage points at 14 cost units under the trainable-parameter accounting. Neither mechanism caused forgetting: retention-suite error fell by 2.0 percentage points under both, a change of three facts that we attribute to single-seed noise rather than to a genuine improvement. Per unit of trainable-parameter compute, the edit therefore removes considerably more error than the full fine-tune. This is the concrete expression of the argument in Section 5.2 that a displacement confined to a small, sparse subspace purchases plasticity at negligible stability cost. This comparison, however, overstates the edit's compute advantage: the forward-inclusive accounting (5) also charges the edit a forward pass through all parameters and a backward pass down to the edited layer for every processed token, so its advantage under that accounting is smaller. Together these

two mechanisms satisfy the ordering assumed in condition (ii) of Proposition 1: the more aggressive action is both more effective and more expensive.

The low-rank adaptation does not. Although it is the cheapest of the three mechanisms at 2.6 cost units under the trainable-parameter accounting, it raises total error from 17.4 to 24.4 percentage points and induces 11.0 percentage points of forgetting on the static retention suite, with a large spread between the 10-day and 30-day interventions. In the language of Section 5.3, an unreplayed adapter trained at a comparatively high learning rate on the drifting facts alone produces a displacement whose direction is not orthogonal to the high-curvature directions of the pre-update loss on the static facts, so the update lands on the steep part of the stability–plasticity frontier; the fact that the adapter is low-rank bounds $\|\Delta\|_0$ but does not by itself constrain the Fisher-weighted norm $\Delta^\top F \Delta$ in (15). This is an empirical instance of the caveat raised in Section 6.5: parameter efficiency is not the same as low forgetting. The measured costs also break the nominal order of (18) in a second way, since the PEFT action is cheaper than the knowledge edit rather than more expensive. The LoRA action is nevertheless harmless to the policy, because it is dominated by the no-op: it costs more, forgets more, and leaves the model with a higher total error than doing nothing. Since the relative value function is non-decreasing in the degradation index (Proposition 1, whose monotonicity part requires only condition (i)), an action whose one-step cost is higher and whose next state is stochastically larger than those of the no-op is never strictly preferred to it, so an MDP instantiated with the measured parameters never needs to select LoRA. Removing it leaves the ordered set no-op $\prec$ localized edit $\prec$ full fine-tune, which satisfies condition (ii). We report this configuration as measured rather than tuning it away, because it illustrates that the ordering in Proposition 1 is a property to be verified for each candidate mechanism from measurements of the kind in Table 4, not an assumption that can be made on the basis of the mechanism's name.

The case study validates the framework at the level of its measured inputs—affine staleness growth, a retrieval-absorbed share equal to retrieval coverage, and a cost/effectiveness/forgetting profile per mechanism that can be read directly into the action set of Section 6—on a small model with synthetic drift. Validation at the level of the scheduling decision itself, comparing periodic retraining with the threshold policy of Proposition 1 in closed loop over a long horizon, and replication over multiple seeds and larger models, remain to be done and are discussed in Section 10.5.

8.5. Summary of Findings

The analytical and numerical results of Sections 4–8 can be summarized as six findings. First, under Poisson drift arrivals the optimal interval between retraining events grows with the square root of the retraining cost and shrinks with the square root of the drift intensity, while the tolerated staleness level grows with both (Section 4.2). Second, whether a memoryless threshold policy is adequate is hypothesized to depend on the aging property of the drift-arrival distribution, with opposite prescriptions for a long quiet interval under DMRL and under IMRL (Sections 4.3 and 8.3). Third, forgetting can be expressed as a Fisher-weighted quadratic form in the parameter displacement, so the stability–plasticity trade-off becomes a Pareto frontier whose steepness depends on the alignment between the update direction and the high-curvature directions of the pre-update loss; lower forgetting per event lowers the optimal cost rate and shortens the optimal interval (Section 5). Fourth, under stochastic-monotonicity and submodularity conditions on the transition kernel, an optimal policy over a graduated menu of maintenance actions escalates through a sequence of degradation thresholds (Proposition 1). Fifth, retrieval augmentation lengthens the optimal interval between parametric updates by the factor $1/\sqrt{1-\bar{\phi}/\gamma}$, dominates parametric maintenance alone exactly when its upkeep cost satisfies (24), and cannot

substitute for parametric maintenance when the drift is not retrievable (Section 7). Sixth, on a public 70M-parameter model with synthetic drift, measured staleness grows affinely with a retrieval-absorbed share equal to retrieval coverage, a localized edit removes more error per unit of trainable-parameter compute than full fine-tuning, and an unreplayed low-rank adaptation violates the action ordering of Proposition 1 and is dominated by the no-op (Section 8.4). The first five findings are derived from the model and illustrated with stylized parameters; the sixth is a controlled measurement on a small model rather than a validation against deployment data.

9. Managerial Implications

The framework is stated in the language of stochastic control, but its results answer questions that managers responsible for deployed LLMs face directly: how much to budget for maintenance, how often to update, which update mechanisms to adopt, how much behavioral regression to tolerate, and whether to invest in retrieval. This section translates the results of Sections 4–8 into guidelines for those decisions. Table 5 summarizes the mapping.

Table 5. Managerial decisions and the results that inform them.

Managerial Decision	Result in This Paper	Practical Guideline
How often to update the model	Square-root law (12)	Set the cadence from measured drift and update cost; a tenfold rise in update cost justifies only about a threefold longer interval
Which update mechanisms to adopt	Proposition 1; Table 4	Admit a mechanism to the maintenance menu only after measuring its cost, effectiveness, and forgetting
How much regression to tolerate	Forgetting constraint (15); optimum (17)	Set the forgetting tolerance and its cost explicitly as governance parameters
Whether to invest in retrieval	Blending condition (24)	Invest when retrieval upkeep is below the saving it creates, and only for drift that retrieval can absorb
What to monitor	Estimation procedure of Section 4.4	Track staleness and retention error on fixed evaluation suites across every maintenance cycle

9.1. Treat Model Maintenance as a Budgeted Operations Decision

The first implication is organizational rather than technical. In many organizations, the decision to update a deployed model is made by the team that built it, on a calendar inherited from software release cycles or in response to a visible incident. The objective (7) shows that the decision in fact trades off three costs that are usually owned by different parts of the organization: the operational loss from an outdated model, which falls on the business units that use it; the compute and engineering cost of an update, which falls on the platform or MLOps team; and the cost of behavioral regressions, which falls on risk, compliance, and customer-facing functions. The coefficients α and β and the tolerance $G_{\max}$ are therefore not technical hyperparameters but statements of organizational priorities. Making them explicit, and assigning ownership of each, turns model maintenance from an engineering routine into a budgeted operations decision that can be reviewed and defended in the same way as other IT maintenance expenditures.

9.2. Size the Maintenance Cadence with the Square-Root Rule

The square-root law of Section 4.2 gives managers a simple rule for budgeting. Because the optimal interval grows only with the square root of the update cost, moving to a model

that is ten times more expensive to update justifies an update interval only about three times longer, not ten times longer. Conversely, if monitoring shows that the environment around the model is changing four times faster than before, the interval should be halved, not quartered. Two common heuristics are therefore miscalibrated: stretching the update cycle in proportion to update cost, which leaves expensive models stale for too long, and matching the update frequency to the drift rate, which overreacts to changes in the environment. The same law also bounds the value of negotiating lower update costs, for example with a cloud provider or model vendor: halving the cost of an update lowers the optimal cost rate by only about 29%, because the organization responds by updating more often.

9.3. Build a Graduated Maintenance Menu and Verify Each Mechanism

Proposition 1 supports a policy of escalation: when degradation is small, apply the lightest mechanism that restores the model, and reserve full retraining for high degradation. For managers, this means that maintenance capability should be built as a menu—targeted knowledge editing, parameter-efficient fine-tuning, and full retraining—with escalation thresholds agreed in advance, rather than as a single retraining pipeline. The case study adds an important qualification. A mechanism's name does not guarantee its place in the menu: the unreplayed low-rank adapter was the cheapest mechanism tested but made the model worse than doing nothing. Before a mechanism is adopted, it should pass an acceptance test that measures its cost, its error reduction, and its forgetting on a fixed retention suite, in the form of Table 4. This acceptance test is the maintenance counterpart of the regression testing that software organizations already apply to code changes, and it can be embedded in the same change-management process.

9.4. Govern Forgetting Explicitly

Every parameter update risks degrading behavior that users and regulators already rely on, and the alignment tax documented by Ouyang et al. [26] shows that this risk extends to safety and policy behavior. The constrained update problem (15) makes the organization's tolerance for such regressions an explicit quantity, $G_{\max}$, that can be set by risk owners, recorded, and audited, instead of being left implicit in a training hyperparameter. The optimum (17) adds two points that are useful in governance discussions. First, residual forgetting imposes a permanent cost, αG, that no choice of update frequency can remove, so a cheap mechanism that forgets can be more expensive in the long run than a costly one that does not. Second, mechanisms that forget less make each update less costly and therefore justify updating more often, so investment in safer update mechanisms pays off partly through a fresher model rather than through fewer updates.

9.5. Invest in Retrieval Where Drift Is Factual

Retrieval augmentation is often presented as an alternative to retraining. The analysis of Section 7 and the case study support a more precise position. Retrieval absorbs the share of drift that its corpus covers and no more: in the case study, a store covering 60% of the drifting facts absorbed 61% of the staleness growth. It therefore substitutes for parametric maintenance only for drift that consists of facts that can be looked up, not for changes in required reasoning, output formats, or policy norms. Condition (24) gives managers an investment test: maintaining a retrieval store is worthwhile when its upkeep cost per unit time is lower than the reduction in the optimal parametric maintenance cost that it produces. Because that reduction grows with the absorbed share, the coverage of the retrieval corpus over the facts that actually change is the key operational metric for a retrieval investment, and it should be monitored as part of the maintenance program rather than assumed once the system is deployed.

9.6. Invest in Monitoring Before Optimizing

All of the guidelines above depend on estimates of the drift rate, the update costs, and the forgetting of each mechanism, and these can only be obtained from a monitoring program. The estimation procedure of Section 4.4 requires fixed evaluation suites for the drifting knowledge and for retained behavior, applied consistently before and after every maintenance event, and records of when drift was detected. Organizations that change their evaluation methods between maintenance cycles lose the ability to estimate these quantities. Until enough cycles have been observed, managers should rely on the qualitative results—the square-root sensitivity of the interval, the escalation structure, and the partial nature of retrieval substitution—which hold regardless of the exact parameter values, and move to numerically optimized policies as monitoring data accumulate.

10. Limitations and Future Work

10.1. Identifiability and Data Requirements

A practical limitation, distinct from the modeling assumptions of Section 3.7, concerns identifiability: the estimation procedure of Section 4.4 requires repeated observation of full maintenance cycles—staleness accumulating from a reset point, a maintenance event, and a subsequent reset—in order to fit even the simplest affine decay model, and organizations that maintain their models very infrequently, or that have changed maintenance mechanism or monitoring methodology between cycles, may simply lack the historical data needed to estimate γ, let alone the richer hazard-rate structure of Section 4.3, with any precision. In such data-poor settings, the qualitative comparative statics derived analytically in Sections 4–6—for instance, the square-root sensitivity of the optimal interval to cost and to drift intensity—are arguably of more immediate practical value than any point estimate of the optimal threshold itself, since the former are robust, structural predictions while the latter is only as reliable as the (possibly very limited) historical record used to calibrate it.

10.2. Scalar Accuracy and Multi-Task Heterogeneity

Assumption A1 treats task-relevant accuracy $A(\theta, w)$ as a single scalar and adds staleness and forgetting in the degradation index, but a deployed LLM is evaluated by its users along many largely independent axes—factual accuracy, coding ability, safety compliance, stylistic preference—each of which may drift at a different rate and respond differently to a given maintenance mechanism. A vector-valued extension of Sections 3–6, in which staleness and forgetting are tracked per task cluster and the maintenance action space is allowed to target clusters selectively (as targeted knowledge editing already does implicitly [12,13]), is a natural next step and would allow the threshold-policy result of Section 6 to be tested for robustness under a genuinely multi-objective cost structure, in the manner of multi-component maintenance models in reliability theory [30].

10.3. Endogeneity of the World Process

Assumption A2 holds that the world process w_t evolves independently of the deployed model's own outputs. This assumption is already strained by the growing presence of LLM-generated text in the corpora used to evaluate and retrain subsequent models, which introduces a feedback loop between a model's maintenance history and the very drift process it is trying to track; formalizing this feedback—for instance, by allowing the drift-event rate λ in Section 4 to depend on cumulative prior deployment—is left for future work and would connect this paper's renewal-theoretic framework to the broader literature on degradation induced by training on model-generated data [31].

10.4. Parameter Estimation Under Non-Stationarity

The estimation procedure sketched in Section 4.4 assumes that the parameters η, λ, μ_M governing drift are themselves stable across the observation window used to estimate them; if these parameters are themselves slowly time-varying (a plausible scenario, since the rate at which the world changes is not itself constant), a more sophisticated online, possibly Bayesian, estimation scheme would be required, and the fixed-threshold and even the regime-dependent policies of Sections 4–6 would need to be re-derived as adaptive-control rather than fixed-parameter optimal-control problems.

10.5. Empirical Validation

This paper is primarily a mathematical modeling contribution: its numerical illustrations in Sections 8.2 and 8.3 are chosen for interpretability, and the case study of Section 8.4, while it uses a real pretrained model, is a controlled experiment on a 70M-parameter checkpoint with synthetic drift, a single random seed, and a closed set of eight candidate answers per fact, none of which reflect the scale or the open-ended evaluation of a production deployment. The cost comparison in that study also depends on the FLOP accounting used (Section 3.5). A natural and important next step is to instantiate the model—the staleness-decay parameters of Section 4, the Fisher-information-based forgetting model of Section 5, and the action-cost structure of Section 6—against telemetry from an actual production LLM deployment, and to test the central falsifiable predictions of the framework: the square-root relationship between retraining cost and optimal interval (Section 4.2), the diminishing sensitivity of the optimum to drift intensity (Section 8.2), the opposite effects of forgetting and of drift intensity on the optimal interval (Sections 5.4 and 4.2), the threshold-escalation structure of the optimal multi-action policy (Proposition 1), and the partial-substitution relationship between retrieval augmentation and parametric maintenance frequency (Section 7.2).

11. Conclusions

This paper has argued that the maintenance of a deployed large language model—the recurring decision of when, how much, and by which mechanism to update it—is a condition-based maintenance problem for a deteriorating system, and can be given a coherent mathematical treatment that unifies previously separate strands of the continual-learning, concept-drift, and MLOps literatures under a single renewal-reward optimization objective drawn from reliability theory. Modeling knowledge staleness as a shock-and-wear process driven by random drift events, modeling update-induced forgetting as a Fisher-information-weighted constraint on parameter displacement in the tradition of EWC [9] and of imperfect maintenance [1], and modeling the joint scheduling and mechanism-selection decision as an MDP yields both closed-form results in restricted cases—most notably a square-root relationship between retraining cost, drift intensity, and the economically optimal maintenance interval, and the result that lower forgetting per event warrants more frequent maintenance—and a structural theorem establishing that, under stochastic-monotonicity and submodularity conditions, an optimal policy over a graduated menu of maintenance mechanisms takes the form of an escalating sequence of degradation thresholds. The framework further shows that retrieval-augmented generation is properly understood not as an alternative to parametric maintenance but as a feedback compensator that partially and boundedly substitutes for it, and gives a closed-form condition under which the blend is worth its upkeep. A case study on a public pretrained model confirms the affine staleness growth and the retrieval-coverage bound that the framework assumes, and, by exhibiting a parameter-efficient update that violates the monotone action ordering, demonstrates that the framework's conditions are measurable properties of candidate mech-

anisms rather than articles of faith. Beyond its specific formulas, the central claim of this paper is methodological: that treating LLM maintenance as a first-class, jointly optimized stochastic control problem—rather than as a collection of separately tuned heuristics for drift detection, forgetting mitigation, and cost control—both explains qualitative patterns already visible in the empirical literature and yields new, falsifiable predictions that can guide the design of maintenance policies for future deployed systems.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data and code supporting the case study in Section 8.4 are available from the author upon reasonable request.

Conflicts of Interest: The author declares no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

CBM	Condition-based maintenance
DMRL	Decreasing mean residual life
EWC	Elastic weight consolidation
FLOP	Floating-point operations
IMRL	Increasing mean residual life
KME	Knowledge-based model editing
LLM	Large language model
LoRA	Low-rank adaptation
MDP	Markov decision process
MLOps	Machine-learning operations
MLP	Multilayer perceptron
OLS	Ordinary least squares
PEFT	Parameter-efficient fine-tuning
RAG	Retrieval-augmented generation
RLHF	Reinforcement learning from human feedback

References

1. Pham, H.; Wang, H. Imperfect maintenance. *Eur. J. Oper. Res.* **1996**, *94*, 425–438. https://doi.org/10.1016/S0377-2217(96)00099-9
2. Jardine, A.K.S.; Lin, D.; Banjevic, D. A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mech. Syst. Signal Process.* **2006**, *20*, 1483–1510. https://doi.org/10.1016/j.ymssp.2005.09.012
3. van Noortwijk, J.M. A survey of the application of gamma processes in maintenance. *Reliab. Eng. Syst. Saf.* **2009**, *94*, 2–21. https://doi.org/10.1016/j.ress.2007.03.019
4. Wang, H. A survey of maintenance policies of deteriorating systems. *Eur. J. Oper. Res.* **2002**, *139*, 469–489. https://doi.org/10.1016/S0377-2217(01)00197-7
5. Lu, J.; Liu, A.; Dong, F.; Gu, F.; Gama, J.; Zhang, G. Learning under concept drift: A review. *IEEE Trans. Knowl. Data Eng.* **2019**, *31*, 2346–2363. https://doi.org/10.1109/TKDE.2018.2876857
6. Mahadevan, A.; Mathioudakis, M. Cost-effective retraining of machine learning models. *arXiv* **2023**, arXiv:2310.04216. https://doi.org/10.48550/arXiv.2310.04216
7. Hoffmann, J.; Borgeaud, S.; Mensch, A.; Buchatskaya, E.; Cai, T.; Rutherford, E.; de Las Casas, D.; Hendricks, L.A.; Welbl, J.; Clark, A.; et al. Training compute-optimal large language models. *arXiv* **2022**, arXiv:2203.15556. https://doi.org/10.48550/arXiv.2203.15556
8. Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T.B.; Chess, B.; Child, R.; Gray, S.; Radford, A.; Wu, J.; Amodei, D. Scaling laws for neural language models. *arXiv* **2020**, arXiv:2001.08361. https://doi.org/10.48550/arXiv.2001.08361

9. Kirkpatrick, J.; Pascanu, R.; Rabinowitz, N.; Veness, J.; Desjardins, G.; Rusu, A.A.; Milan, K.; Quan, J.; Ramalho, T.; Grabska-Barwinska, A.; et al. Overcoming catastrophic forgetting in neural networks. *Proc. Natl. Acad. Sci. USA* **2017**, *114*, 3521–3526. https://doi.org/10.1073/pnas.1611835114
10. Luo, Y.; Yang, Z.; Meng, F.; Li, Y.; Zhou, J.; Zhang, Y. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv* **2023**, arXiv:2308.08747. https://doi.org/10.48550/arXiv.2308.08747
11. Shi, H.; Xu, Z.; Wang, H.; Qin, W.; Wang, W.; Wang, Y.; Wang, Z.; Ebrahimi, S.; Wang, H. Continual learning of large language models: A comprehensive survey. *arXiv* **2024**, arXiv:2404.16789. https://doi.org/10.48550/arXiv.2404.16789
12. Wang, S.; Zhu, Y.; Liu, H.; Zheng, Z.; Chen, C.; Li, J. Knowledge editing for large language models: A survey. *ACM Comput. Surv.* **2024**, *57*, 1–37. https://doi.org/10.1145/3698590
13. Zhang, N.; Yao, Y.; Tian, B.; Wang, P.; Deng, S.; Wang, M.; Xi, Z.; Mao, S.; Zhang, J.; Ni, Y.; et al. A comprehensive study of knowledge editing for large language models. *arXiv* **2024**, arXiv:2401.01286. https://doi.org/10.48550/arXiv.2401.01286
14. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, M.; Wang, H. Retrieval-augmented generation for large language models: A survey. *arXiv* **2023**, arXiv:2312.10997. https://doi.org/10.48550/arXiv.2312.10997
15. Wu, T.; Luo, L.; Li, Y.-F.; Pan, S.; Vu, T.-T.; Haffari, G. Continual learning for large language models: A survey. *arXiv* **2024**, arXiv:2402.01364. https://doi.org/10.48550/arXiv.2402.01364
16. Williams, E.; Sinha, T.; Lyttle, M.; Kanagasingam, Y. Optimising retraining frequency for a paediatric emergency department admission prediction model: Development and temporal validation using real-world data. *Emerg. Med. Australas.* **2026**, *38*, e70271. https://doi.org/10.1111/1742-6723.70271
17. Barlow, R.E.; Proschan, F. *Mathematical Theory of Reliability*; Society for Industrial and Applied Mathematics: Philadelphia, PA, USA, 1996. https://doi.org/10.1137/1.9781611971194
18. Beytur, H.B.; de Veciana, G.; Vikalo, H.; Chan, K.S. Optimal resource allocation for ML model training and deployment under concept drift. *arXiv* **2025**, arXiv:2512.12816. https://doi.org/10.48550/arXiv.2512.12816
19. Ross, S.M. *Stochastic Processes*, 2nd ed.; John Wiley & Sons: New York, NY, USA, 1996.
20. Bertsekas, D.P. *Dynamic Programming and Optimal Control*, 4th ed.; Athena Scientific: Belmont, MA, USA, 2017; Volume I.
21. Puterman, M.L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*; John Wiley & Sons: New York, NY, USA, 1994. https://doi.org/10.1002/9780470316887
22. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 5998–6008. https://doi.org/10.48550/arXiv.1706.03762
23. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901. https://doi.org/10.48550/arXiv.2005.14165
24. Huszár, F. Note on the quadratic penalties in elastic weight consolidation. *Proc. Natl. Acad. Sci. USA* **2018**, *115*, E2496–E2497. https://doi.org/10.1073/pnas.1717042115
25. Kunstner, F.; Balles, L.; Hennig, P. Limitations of the empirical Fisher approximation for natural gradient descent. *Adv. Neural Inf. Process. Syst.* **2019**, *32*. https://doi.org/10.48550/arXiv.1905.12558
26. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training language models to follow instructions with human feedback. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 27730–27744. https://doi.org/10.52202/068431-2011
27. Poenaru-Olaru, L.; Sallou, J.; Cruz, L.; Rellermeyer, J.S.; van Deursen, A. Sustainable machine learning retraining: Optimizing energy efficiency without compromising accuracy. *arXiv* **2025**, arXiv:2506.13838. https://doi.org/10.48550/arXiv.2506.13838
28. Topkis, D.M. *Supermodularity and Complementarity*; Princeton University Press: Princeton, NJ, USA, 1998. https://doi.org/10.1515/9781400822539
29. Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y.J.; Madotto, A.; Fung, P. Survey of hallucination in natural language generation. *ACM Comput. Surv.* **2023**, *55*, 1–38. https://doi.org/10.1145/3571730
30. Dekker, R.; Wildeman, R.E.; van der Duyn Schouten, F.A. A review of multi-component maintenance models with economic dependence. *Math. Methods Oper. Res.* **1997**, *45*, 411–435. https://doi.org/10.1007/BF01194788
31. Shumailov, I.; Shumaylov, Z.; Zhao, Y.; Papernot, N.; Anderson, R.; Gal, Y. AI models collapse when trained on recursively generated data. *Nature* **2024**, *631*, 755–759. https://doi.org/10.1038/s41586-024-07566-y

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.